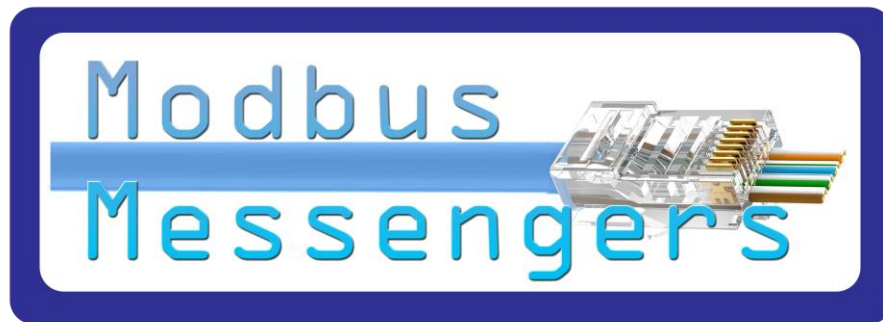


CS4810: Final Design Report

Modbus Messengers

Project 47 –

Modbus ICS in Unreal Engine 5



**UNREAL
ENGINE**

Julia Abdel-Monem

Computer Science
University of Idaho
Moscow, Idaho

abde7751@vandals.uidaho.edu

Hayden Hammes-Buehler

Computer Science
University of Idaho
Moscow, Idaho

bueh7550@vandals.uidaho.edu

Todd Carter

Computer Science
University of Idaho
Moscow, Idaho

cart7982@vandals.uidaho.edu

Amara Jeide

Computer Science
University of Idaho
Moscow, Idaho

jeid1046@vandals.uidaho.edu

Table of Contents

Contents

1. Executive Summary	3
2. Background	3
3. Problem Definition	4
4. Project Plan	5
5. Concepts Considered	7
6. Concept Selection	8
7. System Architecture	9
8. Design Evaluation	9
9. Future Work.....	10
Appendices	11
Appendix A: Modbus Register Types.....	11
Appendix B: Gantt Charts	11
Appendix C: Considered Modbus Libraries	11

1. Executive Summary

The purpose of this project is to create an intuitive solution for Modbus industrial developers using the platform of Unreal Engine 5 (UE). The need for this arises from the prevalence of Modbus ICS in most industrial workspaces. This common nature of Modbus makes it ideal for both educational and practical applications, particularly when utilized for automation and production services. By creating a plugin for Unreal Engine 5, this team will provide a universal platform that is freely accessible for developers to utilize.

By using a widely accessible and constantly maintained platform such as Unreal Engine, we will provide stable access for Modbus integration into the future. This provides cost saving benefits for most applications, as well as reducing the construction times for creating digital twins and industrial prototyping.

2. Background

Digital twins are more than 3D models; they're dynamic, virtual mirrors of physical objects or systems. By using real-time data from a variety of sensors, these digital counterparts allow simulation, prediction, and optimization of the physical world without the risks of real-world trial and error. Professor Conte de Leon teaches a course at the University of Idaho that “examines digital twin technologies for cyber-physical systems and critical infrastructure, including IoT, data pipelines, modeling, and artificial intelligence.” With a tool that allows for seamless integration of Modbus protocols into Unreal Engine, Professor Conte de Leon can focus on implementation of

digital twin philosophies instead of spending a section of the course covering how to implement the protocol into UE. Should he use our plugin, students can expect more time spent designing digital twins and less time spent resolving networking issues that arise from Unreal Engine.

3. Problem Definition

The goals of this project were:

0. Maintain version control with GitHub.

This began with establishing version control through GitHub. Access to GitHub LFS (Large File Storage) was provided by the client for the purpose of development while using Unreal Engine. Version 5.6.2 of Unreal Engine 5 was used for the build process.

1. Use the Unreal Engine 5 Blueprint system.

Using the Unreal Engine 5 Blueprint system was intended to provide future developers with an easier and more intuitive interface. Rather than dealing with code, developers can use the visual system of the Blueprints to construct processes, allowing for much more rapid production.

2. Use an existing Modbus library to manage low-level communications.

Many options for Modbus libraries existed. Using pre-existing Modbus libraries meant relying on code that was often already well-tested and used in other

applications. This increases chances of future maintenance and improves long-term compatibility.

3. Create a plugin that integrates the Modbus library.

The nanoMODBUS library can be integrated into Unreal Engine through a plugin. The Unreal platform allowed for the addition of libraries of code, and this onboard functionality was used as much as possible to improve compatibility.

4. Communicate between Unreal Engine and a Modbus device over TCP/IP.

Using TCP/IP means relying on familiar and heavily tested protocols. Additionally, Unreal Engine 5 is packaged with TCP/IP support, which was also utilized frequently to improve the system connection.

5. Produce a Python-based Modbus simulator.

For the purposes of testing and prototyping, a simple Python emulator was built to provide a stream of Modbus relevant data. This emulator would be run separately and communicate over TCP/IP.

6. Build and test digital twins using the Modbus integration.

Producing simple digital twins allows for testing of the various tools being created through the Modbus plugin. By communication through digital twins across multiple machines, the process of sending and receiving Modbus data could be tested and demonstrated. Using the Unreal Engine 5 Blueprint system was intended to speed the construction process.

4. Project Plan

Agile software development methodologies were utilized throughout the development of the project to balance team members' workload and maintain understanding of client expectations. Organized coding 'sprints' aided in the completion of tasks in a timely manner. See Appendix B for associated Gantt charts. The tasks were as follows:

0. Emulator

Creation of the emulator was scheduled for completion in early December. All basic requirements of the emulator were met by this time, and advanced features were added as needed past that point.

1. Plugin

Development of features for the plugin was broken into linking libraries, integrating TCP/IP, and writing C++ wrappers for library functions. Completion of these tasks was scheduled for late November and early December. By early December, TCP/IP was integrated, and all other tasks were completed by February.

2. Demo

Construction of the demo was scheduled for completion by the end of December. By this point, a simple demo showcasing a valve gauge with capability to read from holding registers was created. Further features were added to the demo

throughout the second semester of work to showcase additional read/write functionality available within the plugin.

3. Testing

Testing was performed incrementally as other features of the project were developed. Section 8 explores testing plans and results in further detail.

4. Documentation

Documentation was created after the implementation was finalized to ensure that minimal changes were required.

5. Concepts Considered

Because implementation within UE was a straightforward process, the C++ Modbus library that we selected was the largest decision tree that we encountered. We considered several open-source C and C++ libraries for Modbus: Mazurel/Modbus, serhmarch/ModbusLib, and debevv/nanoMODBUS. Links to the repositories can be found in Appendix C.

Mazurel/Modbus. This C++ library was created in 2020, with its most recent updates being in October 2025. Unfortunately, TCP implementation was only available on Linux, and most of this project's developers use Windows machines, which put this library out of reasonable implementation.

serhmarch/ModbusLib: This C++ library was the most encompassing of the libraries considered. It supports Modbus functions greater than the scope of what we

need to implement and allows for server and client simulation. However, the library requires compilation before use, which is something that we wanted to stay away from so that we could include the library within our plugin. We also wanted to use Python as a scripting language for our simulator, to allow for faster and easier changes.

debev/nanoMODBUS: This C library was designed to be embedded into a system. Its small size and lack of non-standard library dependencies made it perfect for this implementation.

6. Concept Selection

Ultimately, a library that was lightweight and efficient was deemed to be the most useful. By relying on pre-established libraries, the project would have the advantage of already-tested functionality and pre-existing documentation. Further, the final chosen library would need to be open-source and either freely available or with an available license.

Functionalities that would need to be available in the library included the ability to read and write from each of the Modbus ICS registers and coils. This functionality needed to be either exportable to or otherwise compatible with the Unreal Engine plugin system. Libraries that utilized a programming language that was compatible with Unreal Engine 5 were gathered specifically, which included C, C++, and Python.

Research into several libraries showed that many options were larger than what would be necessary for the scope of the project. Further, the Modbus Messengers team was ready to produce a wrapper that would integrate and upgrade functionality

from a library. This led to the selection of the C library nanoMODBUS, which contained all the functionality that was needed. This library was able to be imported directly into Unreal Engine through the plugin system.

7. System Architecture

Our first goal was the simplest to accomplish — write a wrapper for nanoModbus that could be integrated into UE. This was slightly harder than anticipated, as nanoModbus is written in C and UE primarily uses C++. Even though converting from C to C++ is a straightforward process, the compiler for our plugin struggled to read it as such. After solving that issue, developing a wrapper to implement each read/write function from nanoModbus into UE was very simple, and in that process the functions were tagged as “blueprint-able” functions and UE was able to handle them as such. A “ModbusClient” class was created to manage the socket between UE and the user’s network, as well as to store the functions, addresses, and values between each digital twin and their real-world counterpart.

At the same time, work to create a simple simulated Modbus device using PyModbus. This gave the team the ability to test the read/write functions from within UE without needing a connection to a physical device.

8. Design Evaluation

As per the Design Validation Plan created for this project, the goals of testing for the plugin were set. The set requirements of the plugin were to facilitate integration of

Modbus into UE5, implement blueprints, communicate with real-world devices on TCP using Modbus protocol, and allow for creation of digital twins within the Unreal environment. Testing of these features occurred continuously as they were implemented and updated.

The plugin code was tested to inspect successful integration of Modbus into Unreal Engine. Testing of the code was done by creating digital twins in a sample scene to verify that all functions to read and write from different register types were successful. These tests were also successful in verifying that the creation of digital twins within the engine was possible. Digital twins responded in real time to updates from the emulator. Additionally, functions that allowed the digital twins to write to holding registers were tested. Any changes made to the digital twins were reflected in the registers as displayed by the emulator.

9. Future Work

The plugin is at a point where it can be used to create digital twins without needing to worry about communicating with Modbus. However, further development could focus more on making digital twins easier to develop — such as syncing server registers with local variables. Currently, the user needs to handle every game tick manually. Additionally, creating a system to define registers and their limits to match the server's usage may also be valuable to the creation of digital twins and the use case for this plugin. These two features would probably have a similar scope and duration to the original development of the plugin.

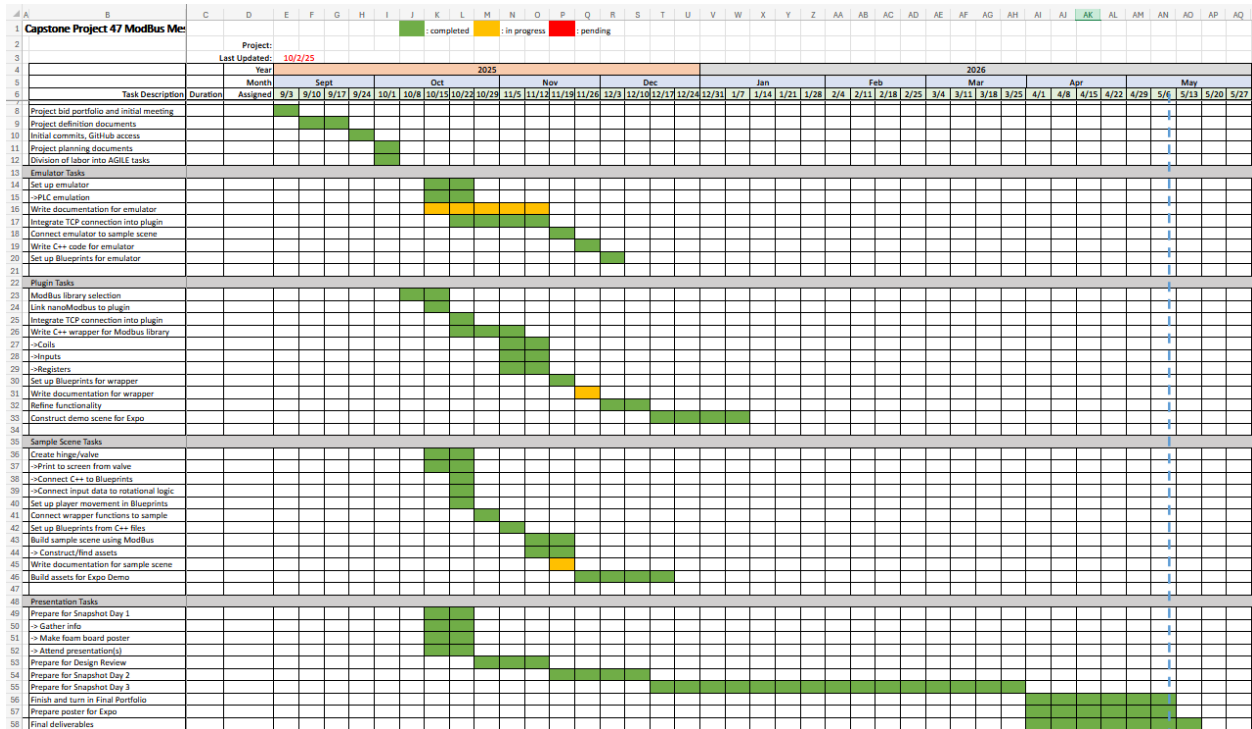
Additionally, validating the plugin with real hardware may be valuable in ensuring that timing and real-world usage matches developer expectations.

Appendices

Appendix A: Modbus Register Types

Register Type	Read/Write	Data Type	Used For
Coils	Read/Write	1-bit	Digital outputs
Discrete Inputs	Read-only	1-bit	Digital inputs
Holding Registers	Read/Write	16-bit	Config, analog output, sensor data
Input Registers	Read-only	16-bit	Sensor values, analog input data

Appendix B: Gantt Charts



Appendix C: Considered Modbus Libraries

Mazurel/Modbus	https://github.com/Mazurel/Modbus
serhmarch/ModbusLib	https://github.com/serhmarch/ModbusLib
debev/nanoMODBUS	https://github.com/debev/nanoMODBUS